Evaluating Recommender Systems

CHAPTER 5

Evaluation Paradigms

There are three primary types of evaluation of recommender systems,

1. user studies,
2. online evaluations,
3. offline evaluations with historical data sets.

The first two types involve users, although they are conducted in slightly different ways.

The main differences between the first two settings lie in how the users are recruited for the studies.

User Studies

- In user studies, test subjects are actively recruited, and they are asked to interact with the recommender system to perform specific tasks.
- Feedback can be collected from the user before and after the interaction, and the system also collects information about their interaction with the recommender system.
- These data are then used to make inferences about the likes or dislikes of the user.
- An important advantage of user studies is that they allow for the collection of information about the user interaction with the system.

Online Evaluation

- Online evaluations are used to test how well recommendation algorithms work in real-world settings with actual users who are using the system naturally (like on an e-commerce site or news app). This method is considered more reliable than lab-based studies because it avoids the biases that can come from specially recruited participants.
- To compare different algorithms, one common method used is A/B testing:
  - Users are randomly divided into two groups — Group A and Group B.
  - Each group is shown recommendations generated by a different algorithm.
  - After some time, the conversion rate (how often users choose or click on the recommended item) is measured for both groups.
  - For example, if users in Group A click on recommended news articles more often than users in Group B, then Algorithm A may be better.
- You can also factor in costs or profits associated with each item to evaluate how much value the recommendations are generating.
- This process is similar to clinical trials in medicine, where different treatments are tested on different patient groups under the same conditions. It's considered the most accurate way to evaluate the long-term impact of recommendation systems, especially for goals like increasing profit or user satisfaction.

Offline Evaluation

- Offline testing uses historical data—such as past user ratings or interactions—to evaluate recommendation algorithms without needing live users. These datasets may include timestamps and other metadata, making them valuable for testing time-sensitive models as well.
- Example: The Netflix Prize dataset is a well-known benchmark used widely for comparing different algorithms.
- The main advantage of the use of historical data sets is that
  - they do not require access to a large user base.
  - Once a data set has been collected, it can be used as a standardized benchmark to compare various algorithms across a variety of settings.
  - Offline methods are among the most popular techniques for testing recommendation algorithms, because standardized frameworks and evaluation measures have been developed for such cases.
- The main disadvantage of offline evaluations is that they do not measure the actual propensity of the user to react to the recommender system in the future.

General Goals of Evaluation Design

General goals include factors such as:

1. Accuracy
2. Coverage
3. Confidence and Trust
4. Diversity,
5. Serendipity,
6. Novelty,
7. Robustness, and
8. Scalability.

Accuracy

Accuracy is a fundamental metric for evaluating recommender systems.Mainly used in systems that predict ratings or rank items.Similar to metrics in regression modeling.

The main components of accuracy evaluation are as follows:

1. **Designing the accuracy evaluation:**

2. **Accuracy metrics:**

1. Designing the accuracy evaluation:

All the observed entries of a ratings matrix cannot be used both for training the model and for accuracy evaluation.

Avoid overfitting: Training and testing must use different data.

Let S = all known ratings Split into:

Training set: $S - E$

Evaluation set: E

Use standard methods: Hold-out validation ,Cross-validation etc

2. Accuracy metrics:

Accuracy metrics are used to evaluate either the prediction accuracy of estimating the ratings of specific user-item combinations or the accuracy of the top k ranking predicted by a recommender system.

Different classes of methods are used for the two cases:

1. Accuracy of estimated rating:the entry-specific error is given by $e_{uj} = \hat{r}_{uj} - r_{uj}$ for user u and item j.

   This error can be leveraged in various ways to compute the overall error over the set E of entries in the ratings matrix on which the evaluation is performed.

   An example is the mean squared error, which is denoted by MSE: $MSE = \frac{\sum_{(u,j) \in E} e_{uj}^2}{|E|}$

   The square-root of the aforementioned quantity is referred to as the root mean squared error, or RMSE.

   $$RMSE = \sqrt{\frac{\sum_{(u,j) \in E} e_{uj}^2}{|E|}}$$

2. Accuracy of estimating rankings

   when the system only recommends top-k items without predicting actual ratings.

   Evaluated using measures like ranking correlation, utility scores, or ROC curves.

Coverage

Even when a recommender system is highly accurate, it may often not be able to ever recommend a certain proportion of the items, or it may not be able to ever recommend to a certain proportion of the users. This measure is referred to as coverage.

This limitation of recommender systems is an artifact of the fact that ratings matrices are sparse. For example, in a rating matrix contains a single entry for each row and each column, then no meaningful recommendations can be made by almost any algorithm.

There are two types of coverage,

user-space coverage and

item-space coverage

Coverage

1. **User-Space Coverage**

- Measures how many users receive at least *k* recommendations.
- If a user has **too little interaction data**, the system may not be able to recommend *k* items.
- Some algorithms (like user-based neighborhood methods) struggle when the user has very few ratings.
- Increasing the size of a neighborhood improves coverage but may reduce accuracy — this creates a **trade-off**.

2. Item-Space Coverage

- Measures how many items can be recommended to at least k users.
- Less commonly used, since systems usually focus on recommending items to users, not users to items.

3. Catalog Coverage

- Looks at how many unique items are recommended across all users.
- Important when systems tend to show the same popular items to everyone.
  Formula:
- where Tu is the top-k list for user u, and n is total number of items.

$$CC = \frac{|\cup_{u=1}^{m} T_u|}{n}$$

Confidence and Trust

Confidence reflects how certain the system is about its predictions, often shown through confidence intervals.

Trust is the user’s belief in those predictions, strengthened when intervals are accurate and consistent.

Novelty

- The novelty of a recommender system evaluates the likelihood of a recommender system to give recommendations to the user that they are not aware of, or that they have not seen before.
- Unseen recommendations often increase the ability of the user to discover important insights into their likes and dislikes that they did not know previously.
- This is more important than discovering items that they were already aware of but they have not rated.

Serendipity

The word *"serendipity"* literally means "lucky discovery." Therefore, *serendipity* is a measure of the level of surprise in successful recommendations.Serendipity in recommender systems refers to the system's ability to surprise users with unexpected and novel recommendations that are not immediately obvious based on the user's past behavior or preferences. In contrast, *novelty* only requires that the user was not aware of the recommendation earlier. *Serendipity* is a stronger condition than novelty.

There are several ways of measuring serendipity in recommender systems.

1. *Online methods:* The recommender system collects user feedback both on the useful ness of a recommendation and its obviousness. The fraction of recommendations that are both useful and non-obvious, is used as a measure of the serendipity.

2. *Offline methods:* One can also use a primitive recommender to generate the information about the obviousness of a recommendation in an automated way. The primitive recommender is typically selected as a content-based recommender, which has a high propensity for recommending obvious items. Then, the fraction of the recommended items in the top-k lists that are correct (i.e., high values of hidden ratings), and are also not recommended by the primitive recommender are determined. This fraction provides a measure of the serendipity.

Diversity

The notion of diversity implies that the set of proposed recommendations within a single recommended list should be as diverse as possible.

For example, consider the case where three movies are recommended to a user in the list of top-3 items. If all three movies are of a particular genre and contain similar actors, then there is little diversity in the recommendations.

If the user dislikes the top choice, then there is a good chance that she might dislike all of them.

Presenting different types of movies can often increase the chance that the user might select one of them.

Note that the diversity is always measured over a set of recommendations, and it is closely related to novelty and serendipity.

Ensuring greater diversity can often increase the novelty and serendipity of the recommendations.

Robustness and Stability

A recommender system is stable and robust when the recommendations are not significantly affected in the presence of attacks such as fake ratings or when the patterns in the data evolve significantly over time.

Scalability

Scalability in recommendation systems refers to the ability of the system to handle increasing amounts of data and users while maintaining performance and accuracy.

A variety of measures are used for determining the scalability of a system:

1. Training time: Most recommender systems require a training phase, which is separate from the testing phase. The overall time required to train a model is used as one of the measures.

2. Prediction time: Once a model has been trained, it is used to determine the top recommendations for a particular customer. It is crucial for the prediction time to be low, because it determines the latency with which the user receives the responses.

3. Memory requirements: When the ratings matrices are large, it is sometimes a challenge to hold the entire matrix in the main memory. In such cases, it is essential to design the algorithm to minimize memory requirements.When the memory requirements become very high, it is difficult to use the systems in large-scale and practical settings.

Design Issues in Offline Recommender Evaluation

The design of recommender evaluation systems is very similar to that of classifier evaluation systems because of the similarity between the recommendation and classification problems.

A common mistake made by analysts in the benchmarking of recommender systems is to use the same data for parameter tuning and for testing.

Such an approach grossly overestimates the accuracy because parameter tuning is a part of training, and the use of test data in the training process leads to overfitting.

To guard against this possibility, the data are often divided into three parts:

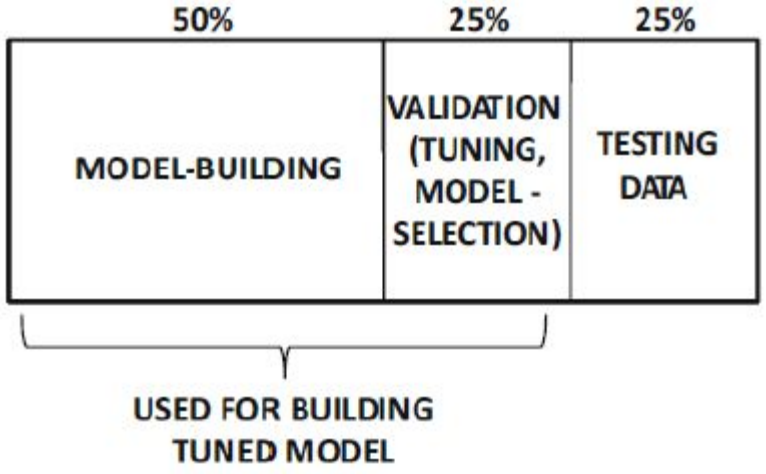
1. Training data
2. Validation data
3. Testing Data

Division of Data

1. **Training data:** This dataset is the primary source for learning and model development. The model uses this data to adjust its parameters and learn underlying patterns, aiming to predict outcomes accurately.

2. **Validation data:** This set is used to evaluate the model's performance during the training process, especially when tuning hyperparameters. It helps determine if the model is generalizing well and prevents it from overfitting to the training data.

3. **Testing Data:** This dataset is separate from the training data and is used to assess the final model's performance on unseen data after training and validation are complete. It provides an unbiased estimate of the model's ability to generalize to new, real-world scenarios



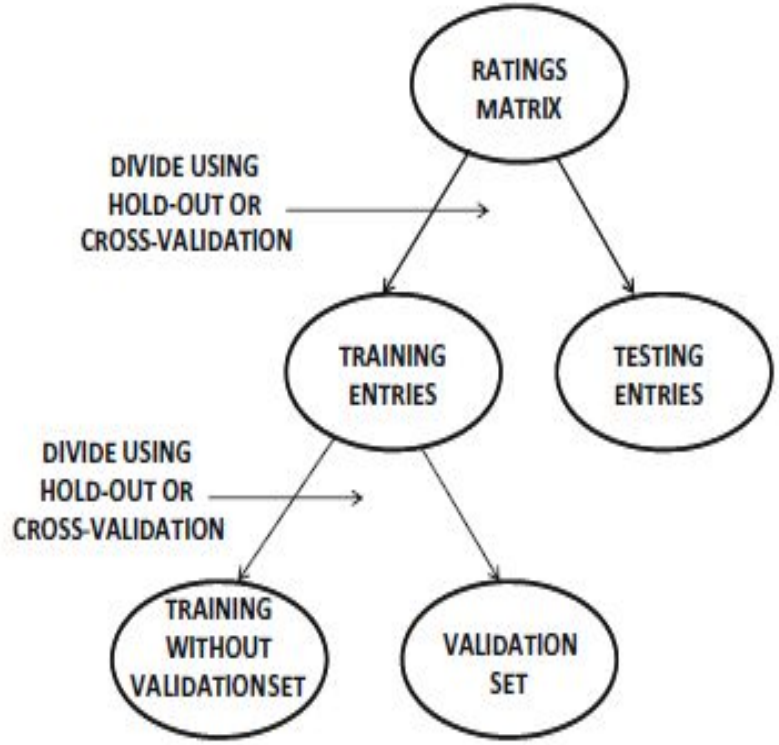
(a) Proportional division of ratings

Segmenting the Ratings for Training and Testing

In practice, real data sets are not pre-partitioned into training, validation, and test data sets. Therefore, it is important to be able to divide the entries of a ratings matrix into these portions automatically.

Most of the available division methods, such as

1. Hold-out and
2. Cross validation,



Hold-out vs. Cross-validation

Hold-out

Hold-out is when you split up your dataset into a ‘train’ and ‘test’ set. The training set is what the model is trained on, and the test set is used to see how well that model performs on unseen data. A common split when using the hold-out method is using 80% of data for training and the remaining 20% of the data for testing.

Cross-validation

Cross-validation or ‘k-fold cross-validation’ is when the dataset is randomly split up into ‘k’ groups. One of the groups is used as the test set and the rest are used as the training set. The model is trained on the training set and scored on the test set. Then the process is repeated until each unique group as been used as the test set.

Accuracy Metrics in Offline Evaluation

Offline evaluation can be performed by

1. Measuring the accuracy of predicting rating values
2. Measuring the accuracy of ranking the recommended items.

1. **Measuring the Accuracy of Ratings Prediction:** the entry-specific error is given by $e_{uj} = \hat{r}_{uj} - r_{uj}$ for user u and item j.

   This error can be leveraged in various ways to compute the overall error over the set E of entries in the ratings matrix on which the evaluation is performed.

   An example is the mean squared error, which is denoted by MSE: $$MSE = \frac{\sum_{(u,j) \in E} e_{uj}^2}{|E|}$$

   The square-root of the aforementioned quantity is referred to as the root mean squared error, or RMSE.

   Other related measures such as the normalized RMSE (NRMSE) and normalized MAE (NMAE) are defined in a similar way, except that each of them is divided by the range rmax − rmin of the ratings:

$$RMSE = \sqrt{\frac{\sum_{(u,j) \in E} e_{uj}^2}{|E|}}$$

$$NRMSE = \frac{RMSE}{r_{max} - r_{min}}$$

$$NMAE = \frac{MAE}{r_{max} - r_{min}}$$

RMSE versus MAE

As the RMSE sums up the squared errors, it is more significantly affected by large error values or outliers. A few badly predicted ratings can significantly ruin the RMSE measure.

In applications where robustness of prediction across various ratings is very important, the RMSE may be a more appropriate measure.

On the other hand, the MAE is a better reflection of the accuracy when the importance of outliers in the evaluation is limited.

The main problem with RMSE is that it is not a true reflection of the average error, and it can sometimes lead to misleading results.

Clearly, the specific choice should depend on the application at hand.

Accuracy Metrics in Offline Evaluation

2. **Measuring the Accuracy of Ranking:** when the system only recommends top-k items without predicting actual ratings.

   Evaluated using measures like

   1. **Ranking correlation,**
   2. **Utility scores, or**
   3. **ROC curves**

Evaluating Ranking via Correlation

- Recommender systems don’t just predict ratings—they rank items and suggest the top-k items to each user.
- To test how good these rankings are, we hide a few known ratings for a user using methods like hold-out or cross-validation.
- For example, if we hide the ratings for items 1, 3, and 5 for a user, we check how well the system predicts the order of these items.
- To compare predicted and actual rankings, rank correlation coefficients are commonly used.
- The two most commonly used rank correlation coefficients are as follows:
  - Spearman rank correlation coefficient
  - Kendall rank correlation coefficient:

Spearman rank correlation coefficient

- The first step is to rank all items from 1 to |Iu|, both for the recommender system prediction and for the ground-truth.
- The Spearman correlation coefficient is simply equal to the Pearson correlation coefficient applied on these ranks.
- The computed value always ranges in (−1, +1), and large positive values are more desirable.
- The Spearman correlation coefficient is specific to user u, and it can then be averaged over all users to obtain a global value.
- Alternatively, the Spearman rank correlation can be computed over all the hidden ratings over all users in one shot, rather than computing user-specific values and averaging them.
- One problem with this approach is that the ground truth will contain many ties, and therefore random tie-breaking might lead to some noise in the evaluation.
- For this purpose, an approach referred to as tie-corrected Spearman is used.

Kendall rank correlation coefficient

For each pair of items j, k ∈ Ii, the following credit C(j, k) is computed by comparing the predicted ranking with the ground-truth ranking of these items:

$$C(j,k) = \begin{cases} +1 & \text{if items } j \text{ and } k \text{ are in the same relative order in} \\ & \text{ground-truth ranking and predicted ranking (concordant)} \\ -1 & \text{if items } j \text{ and } k \text{ are in a different relative order in} \\ & \text{ground-truth ranking and predicted ranking (discordant)} \\ 0 & \text{if items } j \text{ and } k \text{ are tied in either the} \\ & \text{ground-truth ranking or predicted ranking} \end{cases}$$

Then, the Kendall rank correlation coefficient τu, which is specific to user u, is computed

$$\tau_u = \frac{\text{Number of concordant pairs} - \text{Number of discordant pairs}}{\text{Number of pairs in } I_u}$$

Evaluating Ranking via Receiver Operating Characteristic

ROC stands for Receiver Operating Characteristic.

It's a graphical representation of the performance of a binary classification model.

It plots the true positive rate (TPR) against the false positive rate (FPR) at different threshold settings.

TPR is the percentage of correctly predicted positive examples out of all the actual positive examples.

$$TPR = \frac{TP}{P} = \frac{TP}{(TP + FN)} = 1 - FNR$$

FPR is the percentage of incorrectly predicted positive examples out of all the actual negative examples.

$$FPR = \frac{FP}{N} = \frac{FP}{(FP + TN)} = 1 - TNR$$

Precision-Recall curve

- The precision is defined as the percentage of recommended items that truly turn out to be relevant.
- The recall is correspondingly defined as the percentage of ground-truth positives that have been recommended as positive for a list of size t.
- One way of creating a single measure that summarizes both precision and recall is the F1-measure, which is the harmonic mean between the precision and the recall.
- Assume that one selects the top-t set of ranked items to recommend to the user. For any given value t of the size of the recommended list, the set of recommended items is denoted by S(t). Note that |S(t)| = t.

Therefore

$$Precision(t) = 100 \cdot \frac{|\mathcal{S}(t) \cap \mathcal{G}|}{|\mathcal{S}(t)|}$$

$$Recall(t) = 100 \cdot \frac{|\mathcal{S}(t) \cap \mathcal{G}|}{|\mathcal{G}|}$$

$$F_1(t) = \frac{2 \cdot Precision(t) \cdot Recall(t)}{Precision(t) + Recall(t)}$$